4 . Convolutional Neural Network (CNN) from scratch to classify images

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10



(x_train, y_train), (x_test, y_test) = cifar10.load_data()

class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)



x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0


model = models.Sequential([

    layers.Conv2D(
        32, (3, 3),
        activation="relu",
        input_shape=(32, 32, 3)
    ),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64, (3, 3),
        activation="relu"
    ),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64, (3, 3),
        activation="relu"
    )
```

```
    ),

    layers.Flatten(),

    layers.Dense(64, activation="relu"),

    layers.Dense(10, activation="softmax")
])



model.summary()



model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)



history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
)



test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=2
)

print("\nTest Loss:", test_loss)
print("Test Accuracy:", test_accuracy)



plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
```

```
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")

plt.legend()
plt.show()



predictions = model.predict(x_test[:10])
predicted_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(12, 6))

for i in range(10):
    plt.subplot(2, 5, i + 1)

    plt.imshow(x_test[i])

    actual = class_names[int(y_test[i])]
    predicted = class_names[predicted_labels[i]]

    plt.title(
        f"Actual: {actual}\nPred: {predicted}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()



conv_layers = [
    layer for layer in model.layers
    if isinstance(layer, layers.Conv2D)
]

first_conv = conv_layers[0]

filters, biases = first_conv.get_weights()
```

```
print("\nFirst convolution layer filter shape:",
      filters.shape)


filters_min = filters.min()
filters_max = filters.max()

filters = (
    filters - filters_min
) / (filters_max - filters_min)


plt.figure(figsize=(10, 8))

for i in range(min(32, filters.shape[3])):

    plt.subplot(4, 8, i + 1)

    plt.imshow(filters[:, :, :, i])

    plt.axis("off")
    plt.title(f"Filter {i + 1}")

plt.suptitle("Learned Filters - First Convolution Layer")

plt.tight_layout()
plt.show()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-py
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 1979s 12us/step
Training data shape: (50000, 32, 32, 3)
Testing data shape: (10000, 32, 32, 3)
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convoluti
```